SOFTWARE DESIGN SPECIFICATION (SDS)

Project Title: Student Login and Password Management System

Name: Omboko Reginald Paul

Course: BISF (Bachelor of Science in Information Security and Forensics)

Admission Number: 22/04040

Table of Contents

1. Introduction 5
   1.1 Purpose 5
   1.2 Scope 6
   1.3 Definitions, Acronyms, and Abbreviations 6
   1.4 References 7
   1.5 Document Overview 7
2. System Architecture 8
   2.1 Architectural Overview 8
   2.2 Presentation Layer (Frontend) 9
   2.3 Application Layer (Backend) 9
   2.4 Data Layer (Database Architecture) 10
   2.5 Communication Flow 11
   2.6 Security Considerations in Architecture 11
   2.7 Architectural Advantages 11
   2.8 Future Architectural Enhancements (Optional) 11
3. Component Design 12
   3.1 Overview of System Components 12
   3.2 User Interface Component 12
   3.3 Authentication Component 13
   3.4 Password Management Component 14
   3.5 User Management Component 14
   3.6 Session Management Component 15
   3.7 Security and Validation Component 15
   3.8 Database Access Component (Data Access Layer) 16

3.9 Logging and Monitoring Component 16
3.10 Component Interaction Summary 17
3.11 Advantages of Component-Based Design 17
4. Data Design 18
4.1 Database Entities 18
4.1.1 Students Entity 18
4.1.2 Login Activity Entity 19
4.1.3 Password Resets Entity 20
4.1.4 Optional: Password History Entity (Recommended Enhancement) 21
4.3 Data Security 21
4.3.1 Password Storage Security 22
4.3.2 Encryption in Transit 22
4.3.3 Input Validation and Sanitization 22
4.3.4 Access Control 22
4.3.5 Logging and Monitoring 23
4.4 Data Integrity and Backup Strategy 23
4.5 Advantages of the Data Design 23
5. Interface Design 23
5.1 User Interface 24
5.1.1 Design Principles 24
5.1.2 Main User Screens 25
5.1.3 UI Feedback Mechanisms 26
5.1.4 Navigation Design 27
5.1.5 Responsive Design Strategy 27
5.2 External Interfaces 27
5.2.1 Email Service Integration 27
5.2.2 OTP (One-Time Password) Service Integration 28
5.2.3 API Communication Interface 28
5.2.4 Database Interface 29
5.3 Security Considerations in Interface Design 29
5.4 Usability and User Experience Goals 29
5.5 Benefits of the Interface Design 30
6. Security Design 30

6.1 Authentication Security 30
6.1.1 Password Hashing and Storage 30
6.1.2 Secure Session Management 31
6.1.3 Login Attempt Limitation 31
6.1.4 CAPTCHA Integration (Optional) 32
6.1.5 Password Strength Enforcement 32
6.1.6 Account Status Verification 32
6.2 Input Validation 32
6.2.1 Client-Side Validation 32
6.2.2 Server-Side Validation 33
6.2.3 SQL Injection Prevention 33
6.2.4 Cross-Site Scripting (XSS) Protection 33
6.2.5 Cross-Site Request Forgery (CSRF) Protection 33
6.3 Error Handling 34
6.3.1 Secure Error Messages 34
6.3.2 Logging of Errors 34
6.3.3 Exception Handling Strategy 34
6.4 Data Transmission Security 35
6.5 Database Security 35
6.6 Audit and Monitoring 35
6.7 Security Design Benefits 35
7. Performance Considerations 36
7.1 Response Time Optimization 36
7.2 Database Performance Optimization 37
7.2.1 Indexing 37
7.2.2 Efficient Query Design 37
7.2.3 Connection Pooling 37
7.2.4 Transaction Management 38
7.3 Backend Performance Optimization 38
7.3.1 Asynchronous Processing 38
7.3.2 Lightweight Authentication Logic 38
7.3.3 Caching (Optional Enhancement) 38
7.4 Frontend Performance Optimization 38

7.4.1 Efficient Rendering 38
7.4.2 Responsive Design Optimization 39
7.4.3 Client-Side Validation 39
7.5 Scalability Considerations 39
7.5.1 Horizontal Scalability 39
7.5.2 Vertical Scalability 39
7.5.3 Cloud Deployment (Optional) 39
7.6 Concurrency Handling 40
7.7 Monitoring and Performance Testing 40
7.9 Benefits of Performance-Oriented Design 40
8. Design Constraints 41
8.1 Technical Constraints 41
8.1.1 Browser Compatibility 41
8.1.2 Secure Communication Requirement (HTTPS) 42
8.1.3 Database Reliability 42
8.2 Security Constraints 43
8.2.1 Password Storage Standards 43
8.2.2 Access Control Restrictions 43
8.2.3 Regulatory and Institutional Compliance 43
8.3 Hardware Constraints 43
8.4 Network Constraints 44
8.5 Development Constraints 44
8.5.1 Technology Stack Limitations 44
8.5.2 Time Constraints 44
8.5.3 Budget Constraints (If Applicable) 44
8.6 Scalability Constraints 45
8.7 Maintainability Constraints 45
8.8 Risk Constraints 45
8.9 Summary of Design Constraints 45
9. Future Enhancements 46
9.1 Multi-Factor Authentication (MFA) 46
9.2 Biometric Authentication 47
9.3 Enhanced Analytics and Monitoring 47

9.3.1 Login Activity Monitoring 47
9.3.2 Security Analytics 48
9.3.3 Administrative Reports 48
9.4 Integration with Full Student Portal 48
9.5 Single Sign-On (SSO) 49
9.6 Account Recovery Enhancements 49
9.7 Role-Based Access Control (RBAC) 49
9.8 Cloud Deployment and Scalability 50
9.9 User Experience Improvements 50
9.10 Summary of Future Enhancements 50
10. Conclusion 51

1. Introduction

This Software Design Specification (SDS) provides a detailed technical description of the **Student Login and Password Management System**. It defines how the system will be structured, implemented, secured, and maintained. The document serves as a reference for developers, testers, project managers, and system administrators to ensure that the system is built according to clearly defined requirements and design principles.

The system focuses on secure authentication, password management, and controlled access to student resources. It ensures confidentiality, integrity, and availability of user credentials through encryption, validation rules, and controlled access mechanisms.

1.1 Purpose

The purpose of this Software Design Specification is to:

- Provide a clear architectural and technical design of the Student Login and Password Management System.
- Describe system components and how they interact.
- Define database structure and data handling mechanisms.
- Outline user interface design and interaction flow.
- Specify security mechanisms for authentication and password management.
- Provide sufficient information for implementation, testing, deployment, and future maintenance.

This document acts as a technical guideline to reduce ambiguity during development and ensure consistency in system implementation.

1.2 Scope

The Student Login and Password Management System is designed to provide a secure authentication platform for students accessing institutional systems such as portals, learning platforms, and internal services.

The system will include:

1. User Authentication

- Secure login using student ID/email and password.
- Password verification using encrypted storage.
- Session management and automatic session timeout.

2. Password Management

- Password creation with strength validation rules.
- Password reset functionality (via email or OTP verification).
- Password change functionality for authenticated users.
- Prevention of password reuse.

3. Security Controls

- Password hashing using strong cryptographic algorithms.
- Protection against SQL injection and cross-site scripting (XSS).
- Account lockout after multiple failed login attempts.
- Rate limiting for login attempts.
- Encrypted communication using HTTPS.

4. Administrative Controls (Optional Enhancement)

- View login attempts.
- Reset student passwords.
- Monitor suspicious login behaviour.

Out of Scope

- Biometric authentication (fingerprint/face recognition).
- Multi-factor authentication using hardware tokens.
- Integration with external national identity systems.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
SDS	Software Design Specification
UI	User Interface

Term	Meaning
DBMS	Database Management System
OTP	One-Time Password
HTTPS	HyperText Transfer Protocol Secure
Hashing	One-way encryption process used to secure passwords
Session	Temporary server-side storage for user authentication
Authentication	Process of verifying user identity
Authorization	Process of granting access rights
Encryption	Process of converting data into unreadable form

1.4 References

The system design is based on the following standards and best practices:

- IEEE 1016 – Software Design Description Standard
- OWASP (Open Web Application Security Project) Security Guidelines
- ISO/IEC 27001 – Information Security Management
- Institutional IT Security Policy (if applicable)
- Modern Web Development Security Best Practices

These references ensure that the system follows internationally recognized security and software engineering standards.

1.5 Document Overview

This Software Design Specification is structured as follows:

- **Section 1 – Introduction:** Overview, purpose, scope, definitions, and references.
- **Section 2 – System Overview:** General system architecture and design approach.
- **Section 3 – System Architecture Design:** Detailed description of frontend, backend, and database layers.
- **Section 4 – Data Design:** Database schema and relationships.
- **Section 5 – Interface Design:** User interface structure and navigation flow.

- **Section 6 – Security Design:** Authentication mechanisms, encryption methods, and security controls.
- **Section 7 – System Constraints and Assumptions:** Limitations and environmental considerations.

Each section provides sufficient technical detail to guide developers during implementation and ensures the system meets both functional and non-functional requirements.

2. System Architecture

The Student Login and Password Management System adopts a **client-server architecture model**, which separates the system into distinct layers responsible for presentation, processing, and data storage. This architectural approach enhances scalability, security, maintainability, and performance.

The system is logically divided into three major layers:

1. Presentation Layer (Frontend)
2. Application Layer (Backend)
3. Data Layer (Database)

These layers communicate over secure network protocols to ensure safe transmission of sensitive authentication data.

2.1 Architectural Overview

In the client-server architecture:

- The **client (frontend)** is responsible for user interaction and presentation of information.
- The **server (backend)** processes requests, executes business logic, and manages authentication.
- The **database server** stores user credentials and related information securely.

Communication between the client and server occurs via **HTTPS (HyperText Transfer Protocol Secure)**, ensuring encrypted data transmission using SSL/TLS protocols.

This separation of concerns ensures that:

- The user interface can be modified without altering core logic.
- Business rules are centralized in the backend.

- Data is securely managed within the database layer.

2.2 Presentation Layer (Frontend)

The Presentation Layer is responsible for:

- Displaying login forms
- Providing password reset interfaces
- Showing validation messages and error notifications
- Managing user session states (e.g., logged in/logged out view)

Key Responsibilities:

- Collect user credentials (Student ID/Email and Password)
- Perform basic client-side validation (e.g., required fields, password length)
- Send authentication requests to the backend server
- Display authentication results

Technologies (Example Implementation):

- HTML, CSS, JavaScript
- React.js (if applicable)
- Tailwind CSS for styling

The frontend does **not** store passwords or perform authentication logic. It only communicates with the backend through API endpoints.

2.3 Application Layer (Backend)

The Application Layer is the core processing unit of the system. It handles:

- Authentication logic
- Password hashing and verification
- Session management
- Business rule enforcement
- Database communication

Core Functional Modules:

1. Authentication Module

- Verifies user credentials
- Compares entered password with hashed password stored in the database
- Generates secure sessions upon successful login

2. Password Management Module

- Validates password strength
- Hashes passwords before storage

- Handles password reset tokens or OTP verification
- Prevents reuse of previous passwords

3. Security Module

- Protects against SQL injection attacks
- Implements rate limiting for login attempts
- Locks accounts after multiple failed attempts
- Logs suspicious login activities

4. Session Management

- Generates secure session IDs
- Implements automatic session timeout
- Invalidates sessions upon logout

Technologies (Example Implementation):

- Node.js / PHP / Python (depending on implementation)
- RESTful API architecture
- JSON-based communication

The backend ensures that no sensitive operation is performed on the client side.

2.4 Data Layer (Database Architecture)

The Data Layer is responsible for secure storage and retrieval of information.

Main Data Entities:

- Students Table
- Password History Table
- Login Attempts Table
- Password Reset Tokens Table
- Sessions Table (optional)

Database Responsibilities:

- Store hashed passwords (never plain text)
- Maintain user profile information
- Record login timestamps
- Track failed login attempts
- Enforce data integrity constraints

Example Database Technology:

- MySQL / PostgreSQL
- Relational Database Management System (RDBMS)

The database is accessed only through the backend server to prevent unauthorized direct access.

2.5 Communication Flow

The system follows this secure request-response cycle:

- User enters login credentials on frontend.
- Frontend sends credentials to backend API over HTTPS.
- Backend validates credentials.
- Backend queries database for user record.
- Password hash comparison is performed.
- Backend returns authentication response.
- Frontend displays result.

All sensitive communication is encrypted using SSL/TLS certificates.

2.6 Security Considerations in Architecture

The architecture incorporates several security measures:

- HTTPS for encrypted communication
- Password hashing using secure algorithms (e.g., bcrypt, Argon2)
- Prepared statements to prevent SQL injection
- Cross-Site Request Forgery (CSRF) protection
- Rate limiting and account lockout policies
- Secure cookie attributes (HttpOnly, Secure, SameSite)

The layered structure minimizes attack surfaces by ensuring that:

- The database is not exposed directly to clients.
- Authentication logic is handled only in the backend.
- Sensitive data is encrypted both in transit and at rest.

2.7 Architectural Advantages

The client-server architecture provides:

- Scalability – Backend and database can be deployed on separate servers.
- Maintainability – Clear separation of concerns.
- Security – Controlled access to business logic and data.
- Flexibility – Frontend can be upgraded independently.
- Performance Optimization – Caching and load balancing can be added in future enhancements.

2.8 Future Architectural Enhancements (Optional)

The architecture can be extended to support:

- Multi-Factor Authentication (MFA)

- Microservices architecture
- Cloud deployment (AWS, Azure, etc.)
- API gateway and load balancing
- Containerization using Docker

3. Component Design

The Student Login and Password Management System is divided into modular functional components to enhance maintainability, scalability, flexibility, and reusability. Each component is designed with a specific responsibility, following the principle of **separation of concerns**, ensuring that system changes in one module do not negatively affect others.

The modular design allows independent testing, debugging, upgrading, and future expansion of system features.

3.1 Overview of System Components

The system consists of the following major components:

1. User Interface Component
2. Authentication Component
3. Password Management Component
4. User Management Component
5. Session Management Component
6. Security and Validation Component
7. Database Access Component
8. Logging and Monitoring Component

Each component communicates with others through well-defined interfaces and APIs.

3.2 User Interface Component

Purpose:

Handles all interactions between the system and the user.

Responsibilities:

- Display login form
- Display password reset form
- Display password change interface

- Show validation errors and success messages
- Redirect users after authentication

Key Features:

- Client-side form validation
- Responsive design
- Clear error messaging
- Secure input handling (masked password fields)

Design Considerations:

- Should not perform authentication logic
- Must sanitize user inputs before submission
- Should ensure usability and accessibility

This component improves user experience while ensuring secure data collection.

3.3 Authentication Component

Purpose:

Responsible for verifying user identity during login.

Responsibilities:

- Receive login credentials from frontend
- Retrieve user record from database
- Compare entered password with stored hashed password
- Generate authentication response
- Trigger account lockout after multiple failed attempts

Sub-Modules:

- Credential Verification Module
- Account Status Checker
- Login Attempt Tracker

Output:

- Authentication success (create session)
- Authentication failure (error message)

This component ensures that only authorized students gain access to the system.

3.4 Password Management Component

Purpose:

Manages all password-related operations securely.

Responsibilities:

- Validate password strength
- Hash passwords before storage
- Handle password reset requests
- Generate and verify reset tokens or OTP
- Prevent password reuse
- Enforce password expiration policies (optional)

Sub-Modules:

1. Password Strength Validator
2. Password Hashing Module
3. Reset Token Generator
4. Password History Checker

Security Measures:

- Use strong hashing algorithms (bcrypt, Argon2)
- Store only hashed passwords
- Set expiration time for reset tokens

This component ensures compliance with secure password management best practices.

3.5 User Management Component

Purpose:

Handles user account data and profile information.

Responsibilities:

- Store student details (ID, email, name)
- Update profile information
- Activate or deactivate accounts
- Maintain user roles (if applicable)

Features:

- Unique student identification
- Role-based access control (future enhancement)
- Data validation

This component centralizes user data handling and maintains data integrity.

3.6 Session Management Component

Purpose:

Maintains user login state after successful authentication.

Responsibilities:

- Generate secure session ID
- Store session data on server
- Set session expiration time
- Invalidate session upon logout
- Detect inactive sessions

Security Controls:

- Use secure cookies
- Enable HTTP Only and Secure flags
- Implement automatic timeout

This component prevents unauthorized access after login and protects session data.

3.7 Security and Validation Component

Purpose:

Provides system-wide security controls and input validation.

Responsibilities:

- Sanitize user inputs
- Prevent SQL injection
- Protect against Cross-Site Scripting (XSS)
- Enforce rate limiting
- Implement account lockout policy
- Provide CSRF protection

Cross-Cutting Concern:

This component interacts with almost all other modules to ensure consistent security enforcement.

It strengthens the overall system against common web-based attacks.

3.8 Database Access Component (Data Access Layer)

Purpose:

Acts as an intermediary between the application and the database.

Responsibilities:

- Execute database queries
- Use prepared statements
- Handle CRUD operations
- Manage database connections
- Return structured data objects

Benefits:

- Prevents direct database access from other components
- Improves code maintainability
- Simplifies database migration in the future

This abstraction layer ensures secure and efficient data communication.

3.9 Logging and Monitoring Component

Purpose:

Tracks system activities for auditing and security monitoring.

Responsibilities:

- Log successful logins
- Log failed login attempts
- Record password reset activities
- Track suspicious behaviours
- Generate security alerts (optional)

Importance:

- Supports forensic analysis
- Enhances accountability
- Assists in detecting brute-force attacks

3.10 Component Interaction Summary

The interaction flow between components is as follows:

1. User Interface collects login data.
2. Authentication Component verifies credentials.
3. Database Access Component retrieves stored data.
4. Security Component validates inputs.
5. Session Management creates user session.
6. Logging Component records activity.

This modular design ensures that each responsibility is clearly defined, improving system stability and ease of maintenance.

3.11 Advantages of Component-Based Design

- Improved maintainability
- Easier debugging and testing
- Reusability of modules
- Enhanced scalability
- Clear separation of responsibilities
- Reduced system complexity

- Easier integration of new features

4. Data Design

The Student Login and Password Management System uses a **Relational Database Management System (RDBMS)** to store user credentials, profile information, authentication logs, and password recovery data.

A relational database model is selected because it:

- Ensures data integrity through constraints
- Supports structured relationships between entities
- Enables efficient querying and indexing
- Provides strong transactional consistency (ACID properties)
- Enhances security through controlled access mechanisms

The database is accessed exclusively through the backend application layer to prevent direct exposure to clients.

4.1 Database Entities

The system database is designed using normalized tables to eliminate redundancy and maintain consistency. Below are the key entities and their attributes.

4.1.1 Students Entity

Purpose:

Stores registered student account information and authentication credentials.

Table Name:

students

Attributes:

Field Name	Data Type	Description
student_id (PK)	VARCHAR / INT	Unique identifier for each student
full_name	VARCHAR	Student's full name
email	VARCHAR (Unique)	Student email address

Field Name	Data Type	Description
password_hash	VARCHAR	Encrypted password
account_status	ENUM / VARCHAR	Active, Locked, Suspended
created_at	TIMESTAMP	Account creation date
updated_at	TIMESTAMP	Last profile update
last_login	TIMESTAMP	Last successful login

Constraints:

- student_id is the Primary Key (PK).
- email must be unique.
- password_hash cannot be null.
- account_status defaults to "Active".

This entity serves as the core table for authentication and user management.

4.1.2 Login Activity Entity

Purpose:

Records login attempts for monitoring and auditing.

Table Name:

login_activity

Attributes:

Field Name	Data Type	Description
activity_id (PK)	INT (Auto Increment)	Unique record identifier
student_id (FK)	VARCHAR / INT	References Students table
timestamp	TIMESTAMP	Login attempt time
ip_address	VARCHAR	IP address of login request
login_status	VARCHAR	Success or Failed

Field Name	Data Type	Description
device_info	VARCHAR	Optional browser/device details

Relationships:

- student_id is a Foreign Key (FK) referencing students.student_id.
- One student can have many login activity records (One-to-Many relationship).

Purpose of Logging:

- Detect brute-force attacks
- Track suspicious login patterns
- Support audit and compliance requirements

4.1.3 Password Resets Entity

Purpose:

Manages password recovery requests and verification tokens.

Table Name:

password_resets

Attributes:

Field Name	Data Type	Description
reset_id (PK)	INT	Unique identifier
student_id (FK)	VARCHAR / INT	References Students table
reset_token	VARCHAR	Secure random token or OTP
expiry_time	TIMESTAMP	Token expiration time
used_status	BOOLEAN	Indicates whether token is used
created_at	TIMESTAMP	Time token was generated

Security Controls:

- Token must expire after a defined period (e.g., 10–15 minutes).
- Token can only be used once.

- Expired tokens are automatically invalidated.

4.1.4 Optional: Password History Entity (Recommended Enhancement)

To prevent password reuse:

Table Name:

password_history

Field Name	Data Type	Description
history_id (PK)	INT	Unique record
student_id (FK)	VARCHAR / INT	References Students table
password_hash	VARCHAR	Previously used hashed password
changed_at	TIMESTAMP	Password change date

This ensures users cannot reuse recent passwords.

4.2 Data Relationships

The database follows relational integrity rules:

- One Student → Many Login Activities
- One Student → Many Password Reset Records
- One Student → Many Password History Records

Foreign key constraints enforce referential integrity, ensuring no orphan records exist.

Indexes are applied on:

- email
- student_id
- reset_token
- timestamp

to improve query performance.

4.3 Data Security

Data security is a critical design consideration due to the sensitive nature of authentication credentials.

4.3.1 Password Storage Security

Passwords are never stored in plain text.

Instead, secure hashing algorithms such as:

- bcrypt
- Argon2
- PBKDF2

are used to generate irreversible password hashes.

Key Properties of Hashing:

- One-way function (cannot be reversed)
- Salted hashing to prevent rainbow table attacks
- Adaptive cost factor for computational strength

Even database administrators cannot retrieve original passwords.

4.3.2 Encryption in Transit

All data transmitted between client and server is encrypted using:

- HTTPS (SSL/TLS encryption)

This prevents:

- Man-in-the-middle attacks
- Credential interception
- Packet sniffing

4.3.3 Input Validation and Sanitization

All data inputs are validated at both:

- Client side (basic validation)
- Server side (strict validation)

Prepared statements and parameterized queries are used to prevent SQL injection.

4.3.4 Access Control

Database access is restricted by:

- Strong database credentials
- Limited database user privileges

- Firewall restrictions
- Role-based permissions (if applicable)

Only the backend server can interact with the database.

4.3.5 Logging and Monitoring

Sensitive operations are logged, including:

- Failed login attempts
- Password reset requests
- Account lockouts
- Profile updates

Logs help detect unauthorized access attempts and improve system accountability.

4.4 Data Integrity and Backup Strategy

To ensure system reliability:

- Primary and foreign key constraints enforce integrity.
- Transactions ensure consistency.
- Regular automated database backups are scheduled.
- Backup copies are encrypted and stored securely.
- Recovery procedures are documented for disaster recovery.

4.5 Advantages of the Data Design

- Structured and normalized schema
- Strong referential integrity
- Secure password storage
- Efficient query performance
- Scalable design for future enhancements
- Improved auditability and monitoring

5. Interface Design

The Interface Design of the Student Login and Password Management System focuses on **simplicity, clarity, responsiveness, accessibility, and security awareness**. Since the

system primarily handles authentication processes, the user interface must minimize confusion, prevent input errors, and clearly communicate system feedback.

The design follows modern usability principles such as:

- Consistency in layout and styling
- Clear visual hierarchy
- Minimalist form design
- Immediate feedback on user actions
- Mobile-first responsive design
- Accessibility compliance

The interface ensures that users can securely log in, reset passwords, and manage credentials with minimal effort.

5.1 User Interface

The User Interface (UI) represents the Presentation Layer of the system and is responsible for interacting directly with students.

5.1.1 Design Principles

The system interface is designed based on the following principles:

1. Simplicity

- Only essential fields are displayed.
- Avoid unnecessary information during login.
- Clear call-to-action buttons (Login, Reset Password).

2. Clarity

- Labels are clearly associated with input fields.
- Error messages are specific and easy to understand.
- Instructions are concise and visible.

3. Responsiveness

- Fully responsive layout.
- Compatible with desktops, tablets, and mobile devices.
- Adaptive layout for different screen sizes.

4. Accessibility

- High contrast colors for readability.
- Proper font sizes.
- Keyboard navigation support.
- Screen reader compatibility.

5.1.2 Main User Screens

1. Login Page

Features:

- Student ID / Email field
- Password field
- Password visibility toggle (show/hide icon)
- “Forgot Password?” link
- Login button
- Error message display area

Validation:

- Required field validation
- Email format validation
- Clear error messages (e.g., “Invalid credentials”)

Security UI Enhancements:

- Masked password input
- Auto-clear on failed login (optional)
- Account lock notification if applicable

2. Password Reset Page

Features:

- Email or Student ID input field
- Submit button

- Confirmation message
- OTP input (if applicable)

Process:

- User submits email
- System sends reset token or OTP
- User enters verification code
- User sets new password

3. Change Password Page (Authenticated Users)

Features:

- Current password field
- New password field
- Confirm password field
- Password strength indicator
- Submit button

Password Strength Indicator:

- Displays real-time feedback
- Shows criteria (length, uppercase, special characters)

4. Dashboard / Welcome Page (After Login)

Features:

- Welcome message
- Profile summary
- Logout button
- Security alerts (if any)

5.1.3 UI Feedback Mechanisms

To improve user interaction:

- Real-time validation messages

- Success notifications (e.g., “Password updated successfully”)
- Error alerts in visible areas
- Loading indicators during authentication
- Disabled buttons during processing

These feedback mechanisms improve usability and reduce user errors.

5.1.4 Navigation Design

Navigation is kept minimal to enhance focus and security:

- Login → Dashboard
- Dashboard → Change Password
- Login → Forgot Password → Reset
- Logout returns to login page

Navigation avoids unnecessary menu complexity.

5.1.5 Responsive Design Strategy

The system uses:

- Flexible grid layout
- Media queries
- Mobile-first design approach

Benefits:

- Improved accessibility
- Broader device compatibility
- Better user experience

5.2 External Interfaces

The system may integrate with external services to support authentication and recovery processes.

5.2.1 Email Service Integration

Purpose:

- Send password reset links
- Send OTP verification codes

- Send login alerts (optional)

Integration Method:

- SMTP server
- Third-party email API (e.g., SendGrid, Mailgun)
- Institutional mail server

Security Considerations:

- Email tokens must expire
- Emails must not contain plain-text passwords
- Sensitive data must not be embedded in URLs without encryption

5.2.2 OTP (One-Time Password) Service Integration

Purpose:

- Provide additional verification layer during password reset
- Enhance security for sensitive operations

OTP Delivery Methods:

- Email OTP
- SMS OTP (optional enhancement)

Requirements:

- OTP must be randomly generated
- OTP must have expiration time
- OTP must be single-use
- OTP verification attempts must be limited

5.2.3 API Communication Interface

The frontend communicates with the backend through RESTful APIs.

Characteristics:

- JSON request and response format
- Secure HTTPS communication
- Authentication tokens (if using JWT)

- Error handling through HTTP status codes

Example API Endpoints:

- POST /api/login
- POST /api/password-reset
- POST /api/verify-otp
- POST /api/change-password

5.2.4 Database Interface

The backend connects to the relational database using:

- Secure database credentials
- Encrypted connections (if supported)
- Prepared statements
- Connection pooling

Direct database access from the frontend is strictly prohibited.

5.3 Security Considerations in Interface Design

To enhance security at the interface level:

- Password fields are masked by default.
- No sensitive information is displayed in error messages.
- Session timeout automatically logs out inactive users.
- CSRF protection tokens are included in forms.
- Rate limiting prevents brute-force attacks.

5.4 Usability and User Experience Goals

The interface aims to achieve:

- Fast authentication process
- Minimal cognitive load
- Clear security awareness
- Reduced login errors
- Seamless password recovery process

5.5 Benefits of the Interface Design

- Improved user satisfaction
- Reduced login failures
- Increased security awareness
- Cross-device compatibility
- Easy future UI enhancements

6. Security Design

Security is a core requirement of the Student Login and Password Management System. Since the system handles sensitive authentication data such as user credentials and account information, security mechanisms are integrated across all architectural layers — including the frontend, backend, database, and communication channels.

The security design follows internationally recognized best practices such as:

- Defense-in-depth strategy
- Least privilege principle
- Secure-by-design approach
- OWASP security guidelines
- Data confidentiality, integrity, and availability (CIA triad)

Security is implemented proactively rather than reactively to minimize risks of unauthorized access, data breaches, and system compromise.

6.1 Authentication Security

Authentication security ensures that only legitimate users can access the system and that credentials are protected at all times.

6.1.1 Password Hashing and Storage

Passwords are never stored in plain text.

Instead, secure hashing algorithms such as:

- bcrypt
- Argon2

- PBKDF2

are used to generate irreversible password hashes.

Key Security Features:

- One-way hashing (cannot be reversed)
- Automatic salting to prevent rainbow table attacks
- Configurable cost factor to increase computational difficulty
- Unique hash per password

Even if the database is compromised, attackers cannot retrieve original passwords.

6.1.2 Secure Session Management

After successful authentication, a secure session is created.

Session Security Measures:

- Randomly generated session identifiers
- Secure cookies with:
    - HTTP Only flag
    - Secure flag (HTTPS only)
    - SameSite attribute
- Automatic session timeout after inactivity
- Session invalidation upon logout
- Session regeneration after login to prevent session fixation attacks

These controls prevent session hijacking and unauthorized reuse.

6.1.3 Login Attempt Limitation

To prevent brute-force attacks, the system enforces:

- Maximum failed login attempts (e.g., 5 attempts)
- Temporary account lockout after exceeding limit
- Progressive delay between login attempts (optional)
- Logging of failed attempts

This mechanism significantly reduces automated password-guessing attacks.

6.1.4 CAPTCHA Integration (Optional)

To further enhance security, CAPTCHA may be enabled:

- After multiple failed login attempts
- During password reset requests

This prevents automated bots from abusing authentication endpoints.

6.1.5 Password Strength Enforcement

The system enforces strong password policies:

- Minimum character length
- Uppercase and lowercase letters
- Numbers and special characters
- Prevention of commonly used passwords

A real-time password strength indicator guides users in creating secure passwords.

6.1.6 Account Status Verification

Before authentication succeeds, the system verifies:

- Account is active
- Account is not locked
- Account is not suspended

This prevents access from compromised or restricted accounts.

6.2 Input Validation

Input validation is implemented at both the client side and server side to prevent malicious input and ensure data integrity.

6.2.1 Client-Side Validation

Client-side validation improves user experience by:

- Checking required fields
- Validating email format
- Confirming password match
- Enforcing password strength rules

However, client-side validation is not trusted alone and must be reinforced server-side.

6.2.2 Server-Side Validation

All inputs received by the backend are:

- Strictly validated
- Sanitized
- Type-checked
- Length-restricted

Server-side validation ensures that malicious input cannot bypass frontend controls.

6.2.3 SQL Injection Prevention

The system prevents SQL injection by:

- Using prepared statements
- Using parameterized queries
- Avoiding dynamic query construction
- Escaping special characters

Database queries never directly concatenate raw user input.

6.2.4 Cross-Site Scripting (XSS) Protection

To prevent XSS attacks:

- Output encoding is implemented
- User inputs are sanitized
- HTML special characters are escaped
- Content Security Policy (CSP) headers may be configured

This prevents injection of malicious scripts into the application.

6.2.5 Cross-Site Request Forgery (CSRF) Protection

The system includes:

- CSRF tokens in forms
- Verification of token validity on submission
- SameSite cookie attributes

This prevents unauthorized requests from external sites.

6.3 Error Handling

Error handling is designed to provide useful feedback without exposing internal system details.

6.3.1 Secure Error Messages

The system avoids revealing:

- Database structure
- SQL queries
- Stack traces
- Internal server paths
- System configuration details

Instead, generic messages are shown such as:

- "Invalid credentials."
- "Something went wrong. Please try again."
- "Account temporarily locked."

This prevents attackers from gaining system insights.

6.3.2 Logging of Errors

Detailed error information is:

- Logged securely on the server
- Accessible only to administrators
- Stored with timestamps and context
- Monitored for unusual patterns

Logs are not displayed to users.

6.3.3 Exception Handling Strategy

The system implements structured exception handling to:

- Catch runtime errors
- Prevent application crashes
- Roll back incomplete database transactions
- Maintain system stability

Graceful degradation ensures system reliability even under unexpected conditions.

6.4 Data Transmission Security

All communications between client and server use:

- HTTPS (SSL/TLS encryption)

This ensures:

- Data confidentiality in transit
- Protection against packet sniffing
- Prevention of man-in-the-middle attacks

Unencrypted HTTP connections are automatically redirected to HTTPS.

6.5 Database Security

Database-level security measures include:

- Restricted database user privileges
- Strong database credentials
- Firewall rules limiting access
- Encrypted backups
- Regular database updates and patching

The database is not directly accessible from the public internet.

6.6 Audit and Monitoring

To enhance accountability and traceability, the system logs:

- Successful logins
- Failed login attempts
- Password changes
- Account lockouts
- Suspicious activities

These logs assist in detecting and investigating potential security incidents.

6.7 Security Design Benefits

The implemented security mechanisms provide:

- Protection of user credentials
- Prevention of unauthorized access
- Reduced vulnerability to common web attacks
- Secure session handling
- Controlled error exposure
- Improved trust and system reliability

7. Performance Considerations

The Student Login and Password Management System is designed to support multiple concurrent users while maintaining minimal response times, high reliability, and efficient resource utilization. Since authentication systems are often accessed simultaneously (especially during peak academic periods), performance optimization is a critical design requirement.

The performance strategy focuses on:

- Efficient database operations
- Optimized backend processing
- Responsive frontend rendering
- Scalable system architecture
- Resource management and monitoring

The goal is to ensure that users experience fast login responses, smooth navigation, and minimal delays even under increased load.

7.1 Response Time Optimization

The system is designed to maintain low latency during:

- User login
- Password reset requests
- OTP verification
- Profile updates

Target Performance Metrics (Example Benchmarks):

- Login response time: ≤ 2 seconds
- Password reset processing: ≤ 3 seconds
- Page load time: ≤ 2 seconds

Performance is optimized by minimizing unnecessary processing and reducing server overhead.

7.2 Database Performance Optimization

Since authentication relies heavily on database queries, the following strategies are implemented:

7.2.1 Indexing

Indexes are created on frequently queried fields such as:

- student_id
- email
- reset_token
- timestamp

Indexes significantly reduce search time and improve login verification speed.

7.2.2 Efficient Query Design

- Use of optimized SELECT queries
- Avoid unnecessary joins
- Retrieval of only required columns
- Use of parameterized queries
- Avoid full table scans

This reduces database load and improves response time.

7.2.3 Connection Pooling

The backend may implement connection pooling to:

- Reuse database connections
- Reduce connection overhead
- Improve scalability

7.2.4 Transaction Management

Transactions are used only when necessary to avoid locking delays and improve concurrency.

7.3 Backend Performance Optimization

The backend is optimized to handle concurrent requests efficiently.

7.3.1 Asynchronous Processing

Where applicable:

- Non-blocking request handling
- Asynchronous I/O operations
- Background processing for email or OTP sending

This prevents slow operations from delaying authentication responses.

7.3.2 Lightweight Authentication Logic

Authentication processing is kept lightweight:

- Direct hash comparison
- Minimal data retrieval
- Immediate response generation

This ensures fast login verification.

7.3.3 Caching (Optional Enhancement)

Caching may be implemented for:

- Frequently accessed configuration settings
- Temporary authentication tokens

This reduces repeated database queries.

7.4 Frontend Performance Optimization

The frontend is optimized for fast rendering and smooth user interaction.

7.4.1 Efficient Rendering

- Minimal DOM manipulation
- Lazy loading of non-critical resources
- Optimized CSS and JavaScript files
- Removal of unused assets

7.4.2 Responsive Design Optimization

- Lightweight UI components
- Optimized image assets
- Minified stylesheets
- Mobile-first design

This ensures fast loading across devices, including smartphones.

7.4.3 Client-Side Validation

Basic validation is performed on the client side to:

- Reduce unnecessary server requests
- Improve user experience
- Minimize backend load

However, server-side validation remains mandatory.

7.5 Scalability Considerations

The system is designed to scale as the number of users increases.

7.5.1 Horizontal Scalability

The backend can be scaled by:

- Deploying multiple server instances
- Using load balancers
- Distributing traffic across servers

7.5.2 Vertical Scalability

Performance can also be improved by:

- Increasing server RAM
- Upgrading CPU resources
- Using faster storage (SSD)

7.5.3 Cloud Deployment (Optional)

Future enhancements may include deployment on cloud infrastructure such as:

- AWS
- Azure

- Google Cloud

Cloud environments allow auto-scaling based on demand.

7.6 Concurrency Handling

To support multiple concurrent users:

- The backend handles parallel requests
- Database transactions are optimized
- Locking mechanisms are minimized
- Session handling is lightweight

This ensures system stability even during peak usage periods.

7.7 Monitoring and Performance Testing

Performance monitoring tools may be used to track:

- Response times
- Server CPU and memory usage
- Database query performance
- Failed login spikes

Load testing can simulate concurrent users to evaluate system stability before deployment.

7.8 Performance Risks and Mitigation

Risk	**Mitigation Strategy**
High login traffic	Load balancing
Slow database queries	Indexing and query optimization
Excessive failed login attempts	Rate limiting
Server overload	Auto-scaling or resource upgrade
Email/OTP delays	Asynchronous processing

7.9 Benefits of Performance-Oriented Design

- Fast authentication response

- Improved user satisfaction
- Reduced server overload
- Efficient resource utilization
- Scalability for institutional growth
- Stable system during peak periods

8. Design Constraints

Design constraints refer to limitations, conditions, and external requirements that influence how the Student Login and Password Management System is designed, implemented, and deployed. These constraints ensure compliance with technical standards, institutional policies, security requirements, and operational environments.

The system must operate within the boundaries of these constraints while still meeting functional and non-functional requirements.

8.1 Technical Constraints

8.1.1 Browser Compatibility

The system must be compatible with modern web browsers to ensure accessibility for all students.

Supported Browsers:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

Requirements:

- Support latest stable browser versions
- Graceful degradation for slightly older versions
- Responsive design for mobile browsers

Limitations:

- The system may not fully support outdated or deprecated browsers.
- Legacy browsers lacking modern security features (e.g., TLS support) may be restricted.

This constraint ensures broad accessibility while maintaining security standards.

8.1.2 Secure Communication Requirement (HTTPS)

All system communication must occur over HTTPS.

Requirements:

- SSL/TLS encryption enabled
- Valid digital certificate
- Automatic redirection from HTTP to HTTPS
- Secure cookie configuration

This constraint is mandatory to:

- Protect credentials in transit
- Prevent man-in-the-middle attacks
- Ensure data confidentiality
- Meet modern web security standards

The system cannot operate over unencrypted HTTP connections.

8.1.3 Database Reliability

The relational database must maintain high reliability and availability.

Requirements:

- ACID-compliant DBMS (e.g., MySQL, PostgreSQL)
- Transaction consistency
- Backup and recovery capability
- Proper indexing and performance optimization

Constraints:

- Database server must have sufficient storage and memory.
- Downtime may impact authentication availability.

- Data corruption must be prevented through integrity constraints.

Database reliability is critical because authentication services depend entirely on stored credential data.

8.2 Security Constraints

8.2.1 Password Storage Standards

The system must:

- Never store plain-text passwords
- Use strong hashing algorithms (bcrypt, Argon2)
- Enforce password strength policies

Failure to meet these standards would compromise system security and violate best practices.

8.2.2 Access Control Restrictions

- Direct database access from the frontend is prohibited.
- Administrative privileges must be limited.
- Least-privilege principle must be enforced.

These constraints prevent unauthorized system manipulation.

8.2.3 Regulatory and Institutional Compliance

The system must comply with:

- Institutional IT security policies
- Data protection regulations (if applicable)
- Academic integrity policies

Sensitive user data must be handled in accordance with privacy standards.

8.3 Hardware Constraints

The system's performance depends on available hardware resources.

Server Constraints:

- Minimum RAM requirements
- Sufficient CPU capacity
- Reliable storage (preferably SSD)

Client Constraints:

- Users must have internet access
- Devices must support modern browsers
- JavaScript must be enabled

Hardware limitations may affect response times and concurrent user capacity.

8.4 Network Constraints

Since the system is web-based, network performance directly impacts usability.

Constraints include:

- Internet bandwidth availability
- Network latency
- Firewall restrictions
- Institutional network policies

High latency or unstable connections may delay login responses.

8.5 Development Constraints

8.5.1 Technology Stack Limitations

The system must use approved technologies such as:

- Specified backend framework (e.g., Node.js, PHP)
- Relational database (MySQL/PostgreSQL)
- Modern frontend framework (if applicable)

Technology choices may restrict certain features or integration options.

8.5.2 Time Constraints

- Development must meet project deadlines.
- Testing and deployment must occur within academic schedules.

Limited development time may influence feature prioritization.

8.5.3 Budget Constraints (If Applicable)

- Hosting costs
- SSL certificate costs

- Email/OTP service subscription costs

Budget limitations may restrict advanced enhancements such as SMS-based OTP.

8.6 Scalability Constraints

The system is initially designed for a specific number of students. If usage grows significantly:

- Additional server resources may be required.
- Load balancing may be necessary.
- Database scaling strategies must be implemented.

Without scalability planning, performance may degrade under heavy load.

8.7 Maintainability Constraints

The system must:

- Use modular architecture
- Follow coding standards
- Provide documentation
- Support future upgrades

Poor maintainability would increase long-term operational costs.

8.8 Risk Constraints

Potential risks include:

- Server downtime
- Security breaches
- Database corruption
- Email service failure
- Power interruptions

Mitigation strategies must be planned during system deployment.

8.9 Summary of Design Constraints

The major constraints affecting system design include:

- Browser compatibility requirements
- Mandatory HTTPS encryption

- Reliable relational database dependency
- Security compliance standards
- Hardware and network limitations
- Development and budget limitations

Understanding these constraints ensures that the system is designed realistically, securely, and sustainably within its operational environment.

9. Future Enhancements

Future enhancements refer to planned or potential improvements that can be implemented after the initial deployment of the Student Login and Password Management System. These enhancements aim to improve security, usability, scalability, and system integration.

Although the current system fulfils core authentication and password management requirements, future upgrades can significantly enhance overall functionality and user experience.

9.1 Multi-Factor Authentication (MFA)

To strengthen security, the system can implement Multi-Factor Authentication.

Description

Multi-Factor Authentication requires users to verify their identity using two or more authentication factors:

- Something they know (password)
- Something they have (OTP via email or SMS)
- Something they are (biometric verification)

Benefits

- Reduces risk of unauthorized access
- Protects against stolen password attacks
- Enhances institutional security standards

Possible Implementation Options

- Email-based One-Time Password (OTP)
- SMS-based OTP

- Authentication apps (e.g., Google Authenticator)
- Push notification verification

MFA would significantly increase system resilience against credential-based attacks.

9.2 Biometric Authentication

Future versions of the system may support biometric login methods.

Description

Biometric authentication verifies identity using unique physical characteristics such as:

- Fingerprint recognition
- Facial recognition
- Device-based biometric authentication (e.g., mobile fingerprint scanner)

Advantages

- Faster login experience
- Improved user convenience
- Enhanced security

Implementation Considerations

- Integration with device-level authentication APIs
- Privacy and data protection compliance
- Secure storage of biometric tokens (not raw biometric data)

Biometric login would be especially beneficial for mobile users accessing the system via smartphones or tablets.

9.3 Enhanced Analytics and Monitoring

The system can incorporate advanced analytics for monitoring and reporting purposes.

Potential Features

9.3.1 Login Activity Monitoring

- Track login frequency
- Detect suspicious login patterns
- Monitor failed login attempts

9.3.2 Security Analytics

- Identify brute-force attack attempts
- Generate alerts for unusual access locations
- Provide risk scoring for accounts

9.3.3 Administrative Reports

- User activity reports
- Password reset statistics
- System usage trends

Benefits

- Improved security oversight
- Better decision-making for administrators
- Early detection of system vulnerabilities

Enhanced analytics would transform the system from a basic login service into a proactive security management tool.

9.4 Integration with Full Student Portal

The authentication system can be integrated into a comprehensive Student Portal.

Potential Integrations

- Academic records access
- Course registration system
- Fee payment portal
- Attendance tracking
- Examination results viewing
- Library system access

Benefits

- Single Sign-On (SSO) functionality
- Seamless user experience
- Centralized student services

Integration would allow students to log in once and access multiple institutional services securely.

9.5 Single Sign-On (SSO)

Future implementation may include Single Sign-On functionality.

Description

SSO allows users to authenticate once and gain access to multiple related systems without re-entering credentials.

Advantages

- Improved user convenience
- Reduced password fatigue
- Simplified account management

This enhancement would be especially useful if the institution operates multiple web-based systems.

9.6 Account Recovery Enhancements

The password recovery system can be further improved by:

- Security question alternatives
- Time-limited reset tokens
- Backup email verification
- Account recovery via registered mobile number

These enhancements would make recovery both secure and user-friendly.

9.7 Role-Based Access Control (RBAC)

Future upgrades may introduce more advanced access control mechanisms.

Description

Role-Based Access Control assigns permissions based on user roles such as:

- Student
- Administrator
- Faculty
- IT Staff

Benefits

- Improved system organization
- Enhanced security
- Controlled data access

RBAC ensures that users only access features relevant to their role.

9.8 Cloud Deployment and Scalability

The system may later be deployed on cloud infrastructure.

Potential Improvements

- Auto-scaling for high traffic
- Load balancing
- High availability configuration
- Distributed database systems

Cloud deployment would improve performance, reliability, and disaster recovery capabilities.

9.9 User Experience Improvements

Future UI/UX improvements may include:

- Dark mode
- Accessibility enhancements (screen reader compatibility)
- Multilingual support
- Personalized dashboard interface

These features would improve accessibility and user satisfaction.

9.10 Summary of Future Enhancements

Potential future improvements include:

- Multi-Factor Authentication (MFA)
- Biometric login integration
- Advanced security analytics
- Full student portal integration

- Single Sign-On (SSO)
- Role-Based Access Control (RBAC)
- Cloud scalability
- Enhanced user interface features

These enhancements will allow the system to evolve from a basic authentication platform into a secure, scalable, and fully integrated institutional access management solution.

10. Conclusion

This SDS provides a structured blueprint for implementing the Student Login and Password Management System, ensuring consistency, security, and maintainability throughout the development lifecycle.